

MAPPE E ALBERI DI RICERCA

INDICE SEZIONE

- ☐ DESCRIZIONE GENERALE ED APPLICAZIONI
- ☐ DEFINIZIONE ED INTERFACCIA
- ☐ IMPLEMENTAZIONI NAIVE
- ☐ TABELLE HASH
- ☐ MAPPE CON TABELLE HASH
- ☐ ALBERI BINARI DI RICERCA
- ☐ MAPPE CON ALBERI BINARI DI RICERCA
- ☐ CONSIDERAZIONI SULL'EFFICIENZA
- ☐ MULTI-WAY SEARCH TREE
- ☐ (2,4)-TREE
- ☐ RED-BLACK TREE
- ☐ MULTIMAPPE
- ☐ RIEPILOGO

DESCRIZIONE GENERALE ED APPLICAZIONI

Una **mappa** è una *collezione di entry* che permette di *ricercare*, *inserire* e *rimuovere* le entry stesse in base alle loro *chiavi* (come un indice).

Le principali applicazioni sono:

- **Database**.
- **Compilatori**: per il *type checking*.
- **Motori di ricerca**.
- **Data analysis**: per esempio per il conteggio di frequenze degli oggetti.

DEFINIZIONE ED INTERFACCIA

MAPPA

Collezione di *entry* con *chiavi distinte* provenienti da un universo U , su cui è definito l'operatore "=", che supporta i metodi *get*, *put* e *remove* (concetto analogo di indice).

```
public interface Map<K,V> {
    int size();
    boolean isEmpty();

    V get (K key);
    V put (K key, V value);
    V remove (K key);

    Iterable<K> keySet();
    Iterable<V> values();
    Iterable<Entry<K,V>> entrySet();
}
```

- `get(K key)`: se esiste la entry (key,x) *restituisce x*, altrimenti restituisce *null*.
- `put(K key, V value)`: se esiste la entry (key, x) *mette value* al posto di x e *restituisce x*, altrimenti inserisce la entry (key,value) e *restituisce null*.
- `remove(K key)`: se esiste la entry (key,x) *rimuove la entry* e *restituisce x*, altrimenti restituisce *null*.
- Gli ultimi 3 metodi restituiscono strutture (*iterable*) contenenti, rispettivamente, le *chiavi*, i *valori* e le *entry* della mappa che possono essere enumerate da iteratori.

🔗 NOTA

La mappa è vista come *associative array* nel senso che la chiave delle entry è usata come un "indice" di accesso alla mappa.

OSSERVAZIONI

🔗 UNIVERSO DELLE CHIAVI

Non è *necessariamente ordinato*. Se lo è, si parla di *mappa ordinata* (Sorted Map) e, in tal caso, sono possibili implementazioni più efficienti al caso pessimo.

🔗 CHIAVI REPLICATE

La mappa assume che le chiavi siano tutte distinte.

Una sua variante, chiamata **Multimap**, permette di avere più entry con la stessa chiave.

Studieremo principalmente implementazioni basate su:

- Tabelle *hash* (mappa).
- Alberi *di ricerca* (mappa ordinata).

Discuteremo come implementare una Multimap.

MAPPE IN JAVA

Nel package *java.util* troviamo:

1. **Interfaccia** `Map<K,V>`: contiene i metodi visti sopra.
Al suo interno è definita una *interfaccia statica* `Map.Entry<K,V>` associata alla classe e non alle singole istanze della classe.
2. **Classe** `HashMap<K,V>`: implementazione di una mappa tramite tabella hash con *separate chaining*.
3. **Classe** `TreeMap<K,V>`: implementazione di una mappa tramite *red-black Tree*.

IMPLEMENTAZIONI NAIVE

CON LISTA NON ORDINATA

Se n è il numero di entry della mappa, la *complessità* di get, remove, put è

$$\Theta(n)$$

Giustificazione: tutti e 3 i metodi *richiedono la ricerca* di una entry con la chiave data e, se tale entry *non è presente*, è necessario scansionarle tutte.

Tale implementazione è *inefficiente in tempo*.

CON ARRAY DI TAGLIA $|U|$

Si assume che U (universo delle chiavi) sia finito e che sia disponibile un *mapping 1:1* tra U e gli indici in $[0, |U| - 1]$, anche se tale mapping potrebbe essere *non banale* da trovare.

La complessità dei 3 metodi principali sarebbe

$$\Theta(1)$$

Tuttavia tale soluzione è *inefficiente in spazio* se $|U|$ è molto più grande del numero di entry nella mappa.

TABELLE HASH

A questo punto, l'obiettivo è quello di ottenere *prestazioni simili* a quelle della soluzione tramite *array* di taglia $|U|$, ma garantendo *anche efficienza in spazio*.

Per farlo, possiamo pensare di *mappare le chiavi* dal loro universo U ad un insieme di taglia $N \ll |U|$ tramite una funzione h .

In particolare, vogliamo le seguenti proprietà:

- N *non troppo più grande* del numero n di *entry* della mappa.
- Un qualsiasi insieme di n chiavi di U deve essere *distribuito uniformemente* tra gli *indici della tabella* dalla funzione h .

Una soluzione di questo tipo si ottiene con una **tabella hash**.

Si tratta di una struttura definita dai seguenti 3 ingredienti principali:

1. **Funzione hash** $h : U = \{\text{chiavi}\} \rightarrow [0, N - 1]$.
2. **Bucket Array** A di capacità N .
 $A[i]$ rappresenta l' i -esimo bucket al quale vengono associate tutte le *entry* $\langle k, v \rangle$ *tali che* $h(k) = i$ per $0 \leq i < N$.
3. Metodo di **risoluzione delle collisioni**, che sono costituite da chiavi distinte associate allo stesso bucket dalla funzione hash.

🔗 NOTA: CONVENZIONE JAVA

$$h : k \xrightarrow{\text{hash code}} \mathbb{Z} \xrightarrow{\text{compression function}} [0, N - 1]$$

FUNZIONE HASH

Usiamo il *bucket* $A[i]$ per memorizzare tutte le entry $\langle k, v \rangle$ con $h(k) = i$ gestendo le *collisioni* con una *struttura ausiliaria*.

La scelta della funzione hash, nelle sue due componenti, diventa cruciale.

In particolare:

- h deve essere *veloce da calcolare*.
- h deve assomigliare il più possibile ad un *processo random* (**uniform hashing**) che associa a ogni chiave in U un intero su $[0, N - 1]$ tale che $\forall k \neq k' \in U$ e

$\forall i, j \in [0, N - 1]$ si abbia:

$$1. P[h(k) = i] = \frac{1}{N}$$

$$2. P[h(k) = i | h(k') = j] = P[h(k) = i] = \frac{1}{N}$$

🔥 SIGNIFICATO DI 1. E 2.

1. La probabilità che k sia assegnata al bucket $A[i]$ è la *stessa per ogni i* : non ci sono bucket privilegiati da h .
2. Sapendo che k' è mappata in $A[j]$, *non deve cambiare la probabilità* che k sia mappata nel bucket $A[i]$: h deve annullare le possibili correlazioni fra le chiavi.

Le prestazioni di una tabella hash sono tanto migliori quanto più la funzione hash assomiglia ad una *funzione random* che soddisfa 1 e 2.

METODO HASHCODE IN JAVA

Il metodo `hashCode()` della classe `Object` restituisce un `int` che dipende dall'indirizzo in memoria dell'oggetto.

⚠️ ATTENZIONE

Ciò significa che non garantisce che l'`hashCode` di un oggetto rimanga inalterato in diverse esecuzioni di un programma o in diverse implementazioni di Java (*non* è un *mapping puro* da U ad int).

HASH PER TIPI NUMERICI

Il metodo `hashCode()` può essere riscritto in vari modi a seconda del tipo delle chiavi, restituendo sempre un `int`.

Vediamo come trasformare tipi numerici in `int` ($\mathbb{Z}$).

- `byte, short, int, char k` $\rightarrow$ `(int)k`
- `float (32 bit) k` $\rightarrow$ `Float.floatToIntBits(k)`
- `long (64 bit) k` $\rightarrow$ `(int)((k >> 32) + (int)k)` (senza il bit shift si perderebbe l'informazione portata dai primi 32 bit più significativi).
- `double (64 bit) k` $\rightarrow$ `Double.doubleToLongBits` $\rightarrow$ `int` (trasformazione prima in `long` e poi in `int` come sopra).

HASH PER STRINGHE

Le cose cambiano quando vogliamo ottenere l'hash code per una stringa di char
 $S = s_0 s_1 \dots s_{k-1}$.

1. Hash code banale:

$$h(S) = \sum_{i=0}^{k-1} s_i$$

Non è un buon hash code: $h(\text{stop}) = h(\text{tops}) = h(\text{spot})$.

2. Polynomial hash code:

$$h(S) = \sum_{i=0}^{k-1} s_i \cdot a^{k-1-i}$$

Per parole inglesi vanno bene $a = 31, 33, 37, 39, 41$. La classe String in Java implementa hashCode in questo modo con $a = 31$.

3. Hash code basato su cyclic shift: si sommano i singoli caratteri applicando dopo ogni addizione un cyclif shift alla somma parziale:

```
Algorithm hashCodeString(S)
1:  $h \leftarrow s_0$ 
2: for  $i \leftarrow 1$  to  $k-1$  do
3:    $h \leftarrow (h < 5) \parallel (h > 27)$  //cyclic shift (5 pos. con bit-wise OR)
4:    $h \leftarrow h + s_i$ 
5: end for
6: return  $h$ 
```

COMPRESSION FUNCTION

"Comprime" il valore int ottenuto da hashCode nell'intervallo $[0, N - 1]$.

Vediamo due esempi di metodi impiegati per la compression function.

DIVISION METHOD

Dato i intero prodotto dall'hash code:

$$i \rightarrow i \bmod N$$

Dove N è la capacità del bucket array.

OSSERVAZIONE

Per una migliore distribuzione degli hash code tra gli indici del bucket array conviene scegliere N **primo** e **distante da una potenza di 2**.

Scelte di N poco adeguate:

- $N = 2^p$: in tal caso $i \bmod N$ equivale a selezionare i primi p bit meno significativi di i , mentre è meglio che la compression function **dipenda da tutti i bit** di i .
- $N = 10^p$: stesso ragionamento, ma in base 10.

MAD METHOD

Metodo **Multiply-Add-Divide** (MAD):

$$i \rightarrow [(ai + b) \bmod p] \bmod N$$

Dove $p > N$ e p **primo**, $a, b \in [0, p - 1]$ scelti casualmente, con $a > 0$.

OSSERVAZIONE

Il metodo MAD, leggermente **più costoso** dal punto di vista computazionale, assicura una **migliore distribuzione** degli hash code tra gli indici del bucket array.

RISOLUZIONE DELLE COLLISIONI

COLLISIONE

$\langle k_1, v_1 \rangle, \langle k_2, v_2 \rangle$ con $k_1 \neq k_2$ e $h(k_1) = h(k_2)$.

Due possibili soluzioni sono le seguenti.

1. **SEPARATE CHAINING**: Ogni **bucket** è visto come una Map **più piccola** implementata tramite **lista**.
2. **OPEN ADDRESSING**: Non fa ricorso a strutture ausiliarie, ma memorizza le entry **direttamente** nelle **celle del bucket array**. In tal modo si risparmia spazio, ma si complica la gestione delle collisioni. Non studieremo questo approccio.

MAPPE CON TABELLE HASH

Consideriamo l'implementazione di una mappa tramite una *tabella hash* (A, h) con *separate chaining*.

IMPLEMENTAZIONE DEI METODI

Metodo **get**.

```
Algorithm get(k)
1: if  $\exists$  entry (k,x) in  $A[h(k)]$  then
2:   return x
3: end if
4: return null
```

Metodo **put**.

```
Algorithm put(k,v)
1: if  $\exists$  entry (k,x) in  $A[h(k)]$  then
2:   sostituisci x con v
3:   return x
4: else
5:   inserisci (k,v) in coda al bucket  $A[h(k)]$ 
6:   incrementa di 1 size della tabella
7:   return null
8: end if
```

Metodo **remove**.

```
Algorithm remove(k)
1: if  $\exists$  entry (k,x) in  $A[h(k)]$  then
2:   rimuovi (k,x) da  $A[h(k)]$ ;
3:   decrementa di 1 size della tabella
4:   return x
5: else
6:   return null
7: end if
```

COMPLESSITA' DEI METODI

Diamo prima di tutto la seguente definizione.

LOAD FACTOR

Per una tabella hash di *capacità* N che memorizza n entry, il **load factor** λ è definito come:

$$\lambda = \frac{n}{N}$$

In altre parole, λ rappresenta la *lunghezza media di un bucket*.

Studieremo:

- Complessità al **caso pessimo**, in funzione del numero di entry nella tabella hash.
- Complessità al **caso medio** (definito meglio dopo), in funzione del load factor della tabella.

Assumeremo sempre che il *calcolo della funzione hash* per un dato valore k della chiave richieda un numero *costante di operazioni* $\Theta(1)$.

CASO PESSIMO

Consideriamo una tabella hash contenente n entry, e si assuma che per una data chiave k il valore $h(k)$ sia calcolabile in tempo costante.

La complessità al caso pessimo di *get, put, remove* è $\Theta(n)$ ed è dominata dalla *ricerca della entry* con chiave k , necessaria per *tutti e tre* i metodi.

- $O(n)$: banale.
- $\Omega(n)$: basta considerare l'istanza in cui *tutte le entry* sono nello *stesso bucket*. In tal caso, se la chiave k non è presente, bisogna scansionare tutte le entry nel bucket $A[h(k)]$.

CASO MEDIO

TEOREMA

Sotto l'ipotesi di **uniform hashing**, in una tabella hash con *separate chaining* e *load factor* λ la complessità al **caso medio** di get, put, remove è

$$O(1 + \lambda)$$

Tale complessità vale per:

1. Qualsiasi chiave k non presente: la media è fatta su *tutti i possibili valori* di $h(k)$, che sono *equiprobabili* sotto l'ipotesi di uniform hashing.

2. Chiave k presente: la media è fatta assumendo k scelta a caso, con probabilità uniforme, tra le chiavi presenti nella tabella.

Per cui, quando $\lambda \in O(1)$, i tre metodi hanno complessità $O(1)$ al caso medio.

✓ DIM. (1) (Chiave non presente)

Chiamiamo n_i il numero di entry presenti nel bucket di indice i , $0 \leq i < N$.

Quindi $\sum_{i=0}^{N-1} n_i = n$.

La complessità dei 3 metodi è dominata da quella richiesta per la *ricerca della chiave*, che è:

$$O\left(\frac{1}{N} \sum_{i=0}^{N-1} (n_i + 1)\right)$$

OSS: $\frac{1}{N}$ serve per fare la media su tutti i possibili bucket in cui può trovarsi k .

$\Theta(n_i + 1)$ è il numero di operazioni necessarie per cercare k nel bucket di indice i .

Si ha che:

$$\frac{1}{N} \sum_{i=0}^{N-1} (n_i + 1) = \frac{1}{N} (n + N) = \frac{n}{N} + 1 = \lambda + 1$$

E concludiamo quindi che la complessità è effettivamente $O(1 + \lambda)$. $\square$

REHASHING

🔥 OSSERVAZIONE

Nella pratica, si cerca di mantenere $\lambda < 0.9$.

In Java, la classe `HashMap` del package `java.util`, che implementa la tabella hash con separate chaining, usa $\lambda \leq 0.75$ di default.

Quando λ *supera* la soglia prefissata, si esgugue un *rehash*:

- Creazione di un *nuovo bucket array* di capacità $N' \geq 2N$.
- Scelta di una *nuova funzione di hash*.
- *Trasferimento* delle entry dalla vecchia alla nuova tabella hash.

🔥 NOTA

1. In un rehash può essere necessario cercare un numero primo $\geq 2N$, che potrebbe essere costoso (ma ignoriamo tale caso).
2. Il *trasferimento* di n entry in una tabella all'altra può essere implementato in *tempo* $\Theta(n)$ al caso pessimo.
3. Dato che un rehash si esegue quando λ supera una data costante, e la capacità del bucket array almeno raddoppia, il rehash di n entry è preceduto da $\Omega(n)$ put senza rehash. Allora il suo costo viene *ammortizzato* da quello aggregato dei put.

Diamo delle giustificazioni per i punti 2 e 3:

✓ 2

Per spostare n entry da una tabella all'altra posso eseguire n inserimenti nella nuova tabella *disabilitando il controllo* se ogni nuova entry è già presente perchè *sappiamo già* a priori che le entry da trasferire hanno *chiavi distinte*.

✓ 3

Consideriamo un generico rehash con cui passiamo dalla taglia N a $N_{new} \geq 2N$. Chiamiamo $n = \lambda N$ il numero di entry da trasferire.

1. Caso 1: quello considerato è il *primo rehash*. Allora le n entry da trasferire sono state tutte inserite in precedenza, senza rehash.
2. Caso 2: quello considerato *non* è il *primo rehash*. Abbiamo una situazione del tipo: $N_{old} \rightarrow N \geq 2N_{old} \rightarrow N_{new} \geq 2N$.

Il primo rehash nello schema ha trasferito $n_{old} = \lambda N_{old}$ entry, mentre il secondo ne ha trasferite $n = \lambda N$.

Per cui, tra i 2 rehash, ci sono stati almeno $n - n_{old}$ inserimenti, e abbiamo:

$$n - n_{old} = \lambda N - \lambda N_{old} \geq \lambda N - \frac{\lambda N}{2} = \lambda \frac{N}{2} = \frac{n}{2} \frac{N}{N} = \frac{n}{2}.$$

Concludiamo quindi che ci sono stati effettivamente stati $\Omega(n)$ inserimenti tra due generici rehash.

RIEPILOGO TABELLE HASH

- *Facile* implementazione.
- *Buone* prestazioni (al caso medio).
- Non richiede che le chiavi vengano da un universo ordinato.

CONTRO

- Complessità *elevata* al caso pessimo.
- *Incertezza* (alea) dovuta alla bontà della *funzione hash*.
- *Spreco* di *spazio* (per mantenere un basso load factor).

ALBERI BINARI DI RICERCA

ALBERO BINARIO DI RICERCA

E' un *albero binario proprio* in cui i nodi *interni* memorizzano *entry* con chiavi provenienti da un *universo ordinato*, tale che, per ogni nodo interno v la cui entry ha chiave k :

- Le chiavi nel *sottoalbero sinistro* di v sono $< k$.
- Le chiavi nel *sottoalbero destro* di v sono $> k$.

OSSERVAZIONI

1. I nodi *foglia* sono *sentinelle* che delimitano i confini dell'albero. Possono essere implementati con puntatori *null* nei loro padri per non sprecare spazio.
2. La visita *inorder* tocca le entry in *ordine non decrescente*.

ALGORITMO DI RICERCA

Vediamo un algoritmo di ricerca su tali alberi.

- **INPUT**: chiave k , nodo $v \in T$.
- **OUTPUT**: nodo di T_v con chiave k (se esiste), oppure *foglia in posizione giusta* per k .

Algorithm TreeSearch(k, v)

```
1: if T.isExternal( $v$ ) or  $v$ .getElement().getKey()= $k$  then
2:   return  $v$ 
```

```

3: end if
4: if k < v.getElement().getKey() then
5:   return TreeSearch(k, T.left(v))
6: else
7:   return TreeSearch(k, T.right(v))
8: end if

```

OSSERVAZIONI

- Una foglia *giusta* per k è una foglia che, se trasformata in nodo interno può contenere una entry con chiave k senza violare le proprietà di ABR.
- Per ricercare in tutto l'albero si parte con $v = T.root()$.

COMPLESSITA'

Sia h l'altezza di T .

Analizziamo $TreeSearch(k, T.root())$ usando l'albero della ricorsione:

- Esso ha $\leq h + 1$ nodi, dato che *ciascuna invocazione* ricorsiva di $TreeSearch$ *scende di un livello* in T .
- Ogni esecuzione esegue $\Theta(1)$ operazioni, oltre ad un eventuale chiamata ricorsiva.
- Esistono istanze per cui $TreeSearch$ scende effettivamente lungo un cammino di lunghezza $\Theta(h)$ (quando restituisce la *foglia più profonda*).

Per cui la complessità è $\Theta(h + 1)$ e, come al solito, diciamo $\Theta(h)$ per semplicità.

ATTENZIONE

Senza *nessuna ipotesi* sul *grado di bilanciamento* di T , h potrebbe essere $\Theta(n)$.

TREE SEARCH ITERATIVA

- **INPUT:** Albero bin. di ricerca T ed una chiave k .
- **OUTPUT:** nodo $w \in T$ contenente una entry con chiave k se esiste, null altrimenti.

```

Algorithm TreeSearch(k,v)
1: w ← T.root()
2: while T.isInternal(w) do
3:   x ← w.getElement().getKey()
4:   if x=k then
5:     return w

```

```

6:   end if
7:   if k < x then
8:     w ← T.left(w)
9:   else
10:    w ← T.right(w)
11:   end if
12: end while
13: return w

```

Anche in questo caso la complessità è $\Theta(h)$:

- $\Theta(1)$ operazioni fuori dal while.
- $\Theta(h)$ iterazioni del while al caso pessimo.
- $\Theta(1)$ operazioni in ciascuna iterazione del while.

MAPPE CON ALBERI BINARI DI RICERCA

IMPLEMENTAZIONE METODI

GET

Algorithm get(k)

```

1: w ← TreeSearch(k, T.root())
2: if T.isExternal(w) then
3:   return null
4: end if
5: return w.getElement().getValue()

```

La complessità è *dominata da TreeSearch*, per cui sarà $\Theta(h)$.

PUT

Algorithm put(k,x)

```

1: w ← TreeSearch(k, T.root())
2: if T.isInternal(w) then
3:   y ← w.getElement().getValue()
4:   sostituisci x a y nella entry w.getElement()
5:   return y
6: end if
7: expandExternal(w, (k,x) )
8: return null

```

Usiamo un metodo `expandExternal(w,e)` che trasforma `w` in nodo interno contenente la entry `e`.

Anche in questo caso la complessità è dominata da quella di `TreeSearch`, per cui è ancora $\Theta(h)$.

⚠ ATTENZIONE

L'*ordine di inserimento* delle entry influenza la *struttura* dell'albero: inserendo le stesse entry in ordini diversi porta ad alberi diversi.

REMOVE

```
Algorithm remove(k)
1: w gets TreeSearch( K,T.root() )
2: if T.isExternal(w) then
3:   return null                                     //Non trovato
4: else
5:   value ← w.getElement().getValue()
6:   decrementa numero entry in T di 1
7:   if (T.isExternal(T.right(w))) then
8:     esegui caso 1 //CASO 1: w interno con almeno 1 figlio foglia
9:   else
10:    esegui caso 2 //CASO 2: w interno con 2 figli interni
11:   end if
12: end if
```

Caso 1:

```
Algorithm get(k)
1:  $u_L \leftarrow T.left(w)$ 
2:  $u_R \leftarrow T.right(w)$ 
3: if T.isExternal( $u_L$ ) then
4:   cancella w e  $u_L$ 
5:   fai salire  $u_R$  al posto di w
6: else
7:   cancella w e  $u_R$ 
8:   fai salire  $u_L$  al posto di w
9: end if
```

🔗 OSS.

1. Se w era radice, la *nuova radice* è il *figlio* messo al *suo posto*.

2. Il metodo funziona correttamente anche se *entrambi i figli* sono *foglie*.

Caso 2:

Algorithm get(k)

- 1: $w \leftarrow$ nodo con chiave max nel sottoalbero sx di w
- 2: cancella y ed il suo figlio destro
- 3: fai salire il figlio sinistro di y al posto di y

🔥 NOTA

La ricerca di y richiede $\Theta(h)$ operazioni, mentre le altre hanno un costo costante.

In totale, la complessità è, *anche* in questo caso, $\Theta(h)$.

CONSIDERAZIONI SULL'EFFICIENZA

Abbiamo visto come implementare una *mappa ordinata* con gli *ABR*, tuttavia questi presentano una *debolezza*.

⚠️ DEBOLEZZA ABR

Se l'albero è *molto sbilanciato*, $h \in \Theta(n)$ (con n entry) e la complessità dei metodi put, get, remove degenera fino a diventare quella di una semplice *lista ordinata*. Ciò significa che *non* vi è *alcun valore aggiunto*, al caso pessimo, rispetto alle liste o alle tabelle hash.

Le soluzioni possibili sono:

- *Ribilanciare* T ogni volta che lo sbilanciamento supera una certa *soglia* prestabilita (*Red-Black tree*).
- Rendere i *nodi più capienti* in modo da assorbire meglio gli effetti di inserimenti e rimozioni che potrebbero sbilanciare un ABR.

Vedremo di seguito *implementazioni efficienti* della *Mappa ordinata* basate su *varianti* degli alberi binari di ricerca e discuteremo come *generalizzare* la mappa per permettere *chiavi duplicate* (Multimappa).

MULTI-WAY SEARCH TREE

📖 MULTI WAY SEARCH TREE

Abbreviato in *MWS-Tree*, è un albero T *ordinato* tale che:

- Ogni nodo *interno* ha ≥ 2 figli.
- Ogni nodo *interno* con $d \geq 2$ figli $v_1, v_2, \dots, v_d$ (*d-node*) soddisfa le seguenti proprietà:
 1. Memorizza $d - 1$ entry: $(k_1, x_1), (k_2, x_2), \dots, (k_{d-1}, x_{d-1})$ dove $k_1 < k_2 < \dots < k_{d-1}$.
 2. Per $1 \leq i \leq d$ vale che la chiave di ogni entry e memorizzata in un nodo di T_{v_i} soddisfa la relazione $k_{i-1} < e.getKey() < k_i$ assumendo $k_0 = -\infty$ e $k_d = +\infty$.

Anche in questo caso, per convenzione le *foglie* sono delle *sentinelle* e non memorizzano entry.

Un *MWS* generalizza sostanzialmente un *albero binario di ricerca*, permettendo ai nodi di contenere più entry, in modo da rendere più agevole il *bilanciamento*.

Un ABR è un *MWS-Tree* in cui ogni nodo è un *2-node*.

🔥 PROPOSIZIONE

Un MWS-Tree che memorizza n entry ha $n + 1$ foglie.

✓ DIM.

Chiamiamo:

- $A = \{ \text{nodi interni di } T \}$
- $B = \{ \text{foglie di } T \}$
- $\forall v \in T$ sia d_v il numero di figli di v . (Se v è foglia, $d_v = 0$ e v non contiene entry, se v è interno $d_v > 0$ e v contiene $d_v - 1$ entry)

Allora abbiamo:

$$n = \sum_{v \in A} (d_v - 1)$$

Sappiamo che $\sum_{v \in T} d_v = |A| + |B| - 1$.

Inoltre vale $\sum_{v \in T} d_v = \sum_{v \in A} d_v$.

Allora:

$$\begin{aligned} n &= \sum_{v \in A} (d_v - 1) = \left(\sum_{v \in A} d_v \right) - |A| = \\ &= |A| + |B| - 1 - |A| = |B| - 1 \end{aligned}$$

Per cui concludiamo che:

$$|B| = n + 1$$

□.

🔗 OSSERVAZIONE

Se *tutti* i nodi *interni* fossero de d -node il MWST sarebbe un albero *d-ario*.
 Detto x il numero di nodi interni ed y il numero di foglie di T sappiamo che $y = (d - 1)x + 1$, che è coerente con la proposizione appena vista dato che l'albero in questione memorizzerebbe $n = (d - 1)x$ entry.

RICERCA SU MWST

Vediamo l'algoritmo di *ricerca* (ricorsivo, prima chiamata effettuata con $v = T.root()$) su un MWST:

- **INPUT:** chiave k , nodo $v \in T$
- **OUTPUT:** noto di T_v contenente una entry con chiave k , se esiste, o foglia in *posizione giusta* per k altrimenti.

```

Algorithm MWTreeSearch( $k, v$ )
1: if  $T.isExternal(v)$  then
2:   return  $v$ 
3: end if
4: trova  $i$  tale che  $k_{i-1} < k \leq k_i$ 
5: if  $k = k_i$  then
6:   return  $v$ 
7: else
8:   return MWTreeSearch( $k, v_i$ )
9: end if
    
```

Dove $(k_1, x_1), \dots, (k_{d-1}, x_{d-1})$ sono le entry del nodo v , con $k_1 < \dots < k_{d-1}$ (assumendo $k_0 = -\infty$ e $k_d = +\infty$), e $v_1, \dots, v_d$ sono i figli di v .

Analizziamone ora la *complessità*, assumendo che le *entry* in un nodo interno siano memorizzate in una *lista ordinata*.

Albero della *ricorsione* per MWTreeSearch($k, T.root()$):

- E' composto da $\leq h + 1$ *chiamate ricorsive* con h altezza di T .
- Il costo associato a *ciascuna chiamata* ricorsiva è $\Theta(d_{max})$ al caso pessimo, dove d_{max} è il massimo numero di figli di un nodo.

La complessità è pertanto:

$$\Theta(h \cdot d_{max})$$

OSSERVAZIONE

La complessità *non migliora* in generale rispetto a quella di TreeSearch per gli *ABR* in quanto i MWST *non hanno limiti* su h e d_{max} , se non quello banale dato da n numero di entry in T .

METODO GET

Vediamo come implementare il metodo `get` su una *mappa* basata su *MWST*.

Algorithm `get(k)`

```
1: w ← MWTreeSearch(k, T.root())
2: if T.isExternal(w) then
3:   return null
4: else
5:   trova e ∈ w tale che e.getKey()=k
6:   return e.getValue()
7: end if
```

La complessità è *dominata* da quella di MWTreeSearch ed è quindi $O(d_{max}h)$.

(2,4)-TREE

(2,4)-TREE

E' un *MWST* tale che:

- Ogni *nodo interno* è un *d-node* con $2 \leq d \leq 4$. (d figli, $d - 1$ entry)
- Tutte le *foglie* hanno la *stessa profondità*.

NOTA

Si può definire in modo analogo anche un (2,3)-Tree.

Il vantaggio del (2,4)-Tree è che si *generalizza* al *B-Tree* che è una struttura dati molto usata per la realizzazione di *indici di memoria secondaria*.

PROPOSIZIONE

Un (2,4)-Tree con $n > 0$ entry ha altezza $\Theta(\log n)$.

COROLLARIO:

In (2,4)-Tree con n entry la complessità di `MWTreeSearch` e del metodo `get` della mappa è $\Theta(\log n)$.

✓ **DIM.**

Sia T un (2,4)-Tree con n entry e altezza h .

Sia m_i il numero di nodi al livello i per $0 \leq i \leq h$.

Si ha che:

$$m_0 = 1$$

$$2 \leq m_1 \leq 4$$

$$2^2 \leq m_2 \leq 4^2$$

$$2^i \leq m_i \leq 4^i$$

$$2^h \leq m_h \leq 4^h$$

Sappiamo, per definizione, che le foglie sono tutti i nodi al livello h .

Inoltre, essendo il (2,4)-Tree un MWST, sappiamo anche le foglie sono $n + 1$.

Allora:

$$2^h \leq m_h = n + 1 \leq 4^h$$

Da cui:

$$h \leq \log_2(n + 1) \leq \log_2(4^h) = 2h$$

Quindi possiamo dire che valgono le seguenti disequazioni:

$$h \leq \log_2(n + 1)$$

$$h \geq \frac{\log_2(n + 1)}{2}$$

Per cui concludiamo che $h \in \Theta(\log n)$.

Il corollario segue immediatamente da questo e dal fatto che $d_{max} \leq 4$ usando l'analisi fatta per il MWST. $\square$

METODO PUT

1. Se la chiave *non è presente*, inserisci la nuova entry $e = (k, x)$ in un *nodo giusto* per k ad altezza 1.
2. Se il nodo in cui è stata inserita e va in *overflow* (ovvero ha 4 entry e 5 figli), invoca il metodo Split che *ripristina* le proprietà del (2,4)-Tree:
 - Sfruttando la *flessibilità* sul *numero di entry* ammissibili in un nodo.
 - Propagando, se necessario, l'*overflow* verso l'*alto* che, se arriva alla *radice*, fa *crescere* di 1 l'*altezza* dell'albero.

Algorithm put(k,x)

```

1: w ← MWTreeSearch(k, T.root())
2: if T.isInternal(w) then           //entry con chiave k interna
3:   e ← entry in w con chiave k
4:   y ← e.getValue()
5:   sostituisci x a y in e
6:   return y
7: end if
8: e ← (k,x)                         //Caso w foglia
9: if T.isRoot(w) then
10:  expandExternal(w,e)              //(*)
11: else
12:  u ← T.parent(w)
13:  inserisci e in u aggiungendo una foglia w'      //(*)
14:  if u è 5-node then
15:    Split(u)
16:  end if
17: end if
18: incrementa il numero di entry in T di 1
19: return null

```

Chiariamo ora i punti evidenziati con (*):

- `expandExternal(w,e)` (caso *w foglia e radice*): inserisce la entry e in w e aggiunge due foglie.
- `inserisci e ...`: inserisce la entry e nel *nodo genitore* e aggiunge una foglia (per mantenere 1 foglia più delle entry). Se si verifica *overflow* si esegue Split.

Vediamo a questo punto in cosa consiste Split:

- **INPUT**: 5-node $u \in T$, unica violazione delle proprietà di (2,4)-Tree
- **OUTPUT**: Ripristino delle proprietà di (2,4)-Tree

Sia $u = (e_1, e_2, e_3, e_4)$ con figli u_1, u_2, u_3, u_4, u_5 (u è il nodo in *overflow*).

Algorithm Split(u)

```

1: u' = (e1, e2)           //figli u1, u2, u3
2: u'' = (e4)              //figli u4, u5

```

```

3: if !T.isRoot(u) then
4:   v ← T.parent(u)
5:   inserisci  $e_3$  in v con figlio sx u' e figlio dx u''
6:   cancella u
7:   if v è 5-node then
8:     Split(v)
9:   end if
10: else
11:   crea una nuova radice contenente  $e_3$  con due figli u' e u''
12:   cancella u
13: end if

```

COMPLESSITA'

Per una *mappa* con n entry implementata tramite un *(2,4)-Tree* la complessità di put è

$$\Theta(\log n)$$

La complessità è infatti *dominata* da MWTreeSearch e da Split (le *altre operazioni* hanno un costo *costante*): abbiamo visto che il primo ha complessità $\Theta(\log n)$, ci rimane quindi solo da studiare Split.

Si tratta di un algoritmo ricorsivo che:

- E' invocato la *prima volta* su un nodo ad *altezza 1*.
- Le invocazioni successive sono *una per livello*, fino ad arrivare *eventualmente* fino alla *radice*: $\implies \Theta(h)$ invocazioni.
- Ciascuna invocazione ha un costo $\Theta(1)$.

Per cui la complessità di Split è $\Theta(\log n)$, e così anche quella di put.

METODO REMOVE

IDEA

1. Si *rimuovono* solo entry in nodi ad *altezza 1*.
2. Se la entry e da rimuovere ad *altezza > 1*, *sostituiscila* con una entry e' ad *altezza 1* e rimuovi quella.
3. Se il nodo ad altezza 1 da cui è stata rimossa una entry va in *underflow* (contiene *0 entry*) *ripristina* le proprietà del MWST:
 - Sfruttando la flessibilità sul numero di entry per nodo.
 - Propagando, se necessario, l'*underflow* verso l'*alto*, che, se arriva alla radice, fa *diminuire di 1 l'altezza* dell'albero.

La rimozione di una entry da un nodo e la gestione dell'eventuale underflow sarà effettuata tramite il metodo ausiliario Delete.

```

Algorithm remove(k)
1:  $w \leftarrow \text{MWTreeSearch}(k, T.\text{root}())$ 
2: if  $T.\text{isExternal}(w)$  then
3:   return null
4: else
5:   trova  $e \in w$  tale che  $e.\text{getKey}()=k$ 
6:    $y \leftarrow e.\text{getValue}()$ 
7:   if  $\text{height}(w)=1$  then
8:     Delete( $e, w$ )
9:   else
10:     $v \leftarrow$  figlio di  $w$  a sx di  $e$ 
11:     $e' \leftarrow$  entry con chiave max in  $T_v$  //  $O(\log n)$  operazioni
12:     $z \leftarrow$  nodo contenente  $e'$  // nodo ad altezza 1
13:    metti una copia di  $e'$  al posto di  $e$  in  $w$ 
14:    Delete( $e', z$ )
15:   end if
16: end if
17: decrementa di 1 il numero di entry in  $T$ 
18: return  $y$ 

```

Vediamo a questo punto come funziona il metodo Delete(e, u) (*ricorsivo*):

- **INPUT:** $u \in T$ con entry e , e con un figlio foglia o vuoto a sx o dx di e .
- **OUTPUT:** rimozione di e da T ripristinando le proprietà di (2,4)-Tree.

Siano A, X i figli di u discriminati da e dove X è foglia o vuoto.

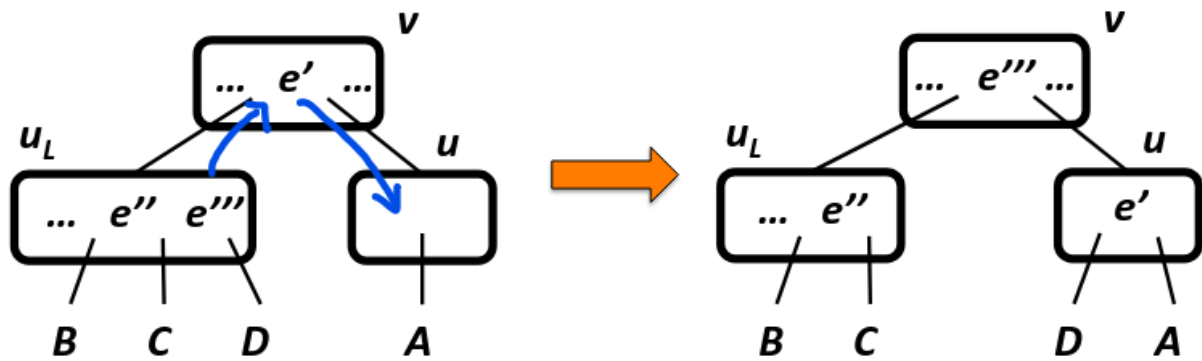
```

Algorithm Delete( $e, u$ )
1: rimuovi  $e$  e  $X$ 
2: if  $u$  non ha altre entry then
3:   CASO 1 :  $u$  radice
4:   CASI 2,3 :  $u$  con un fratello (sx o dx) d-node,  $d=3,4$ 
5:   CASI 4,5 :  $u$  con solo fratelli 2-node
6: end if

```

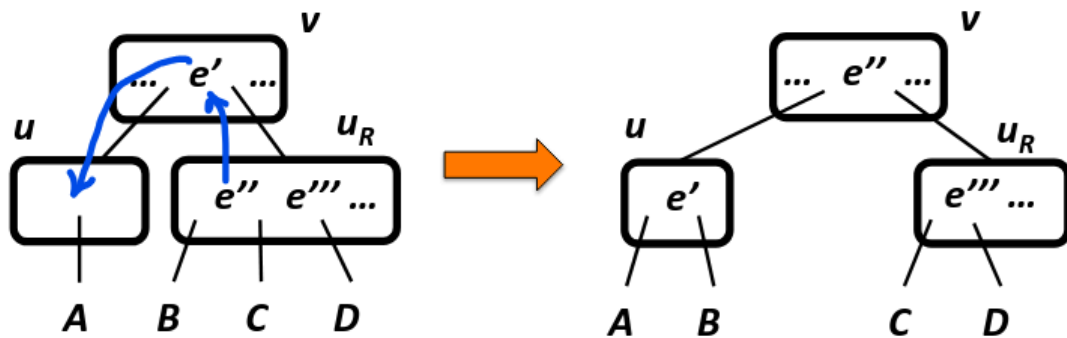
CASO 1: imposta A come *nuova radice* del (2,4)-Tree. In questo caso l'altezza diminuisce di 1.

CASO 2: u ha un fratello u_L a sx che è un d-node con $d \geq 3$.



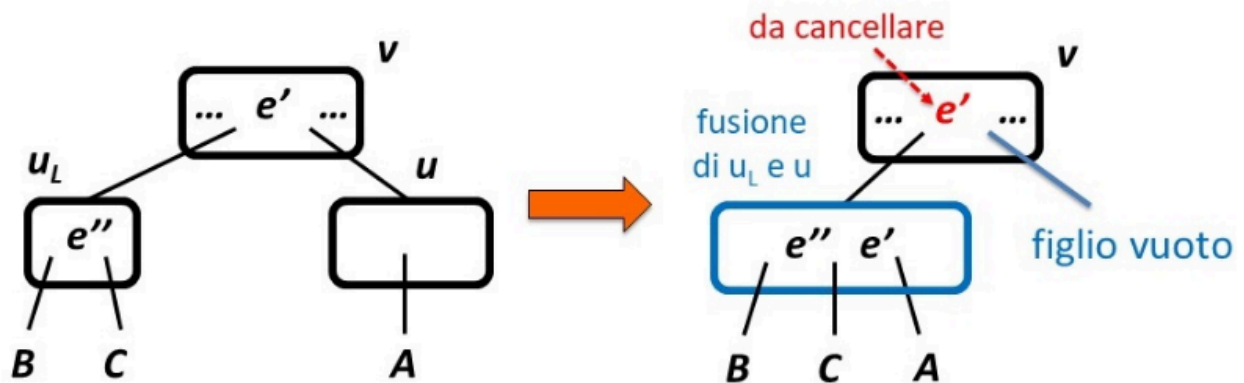
Con questa rotazione Delete *termina l'esecuzione*.

CASO 3: u ha un fratello u_R a dx che è un d-node con $d \geq 3$.



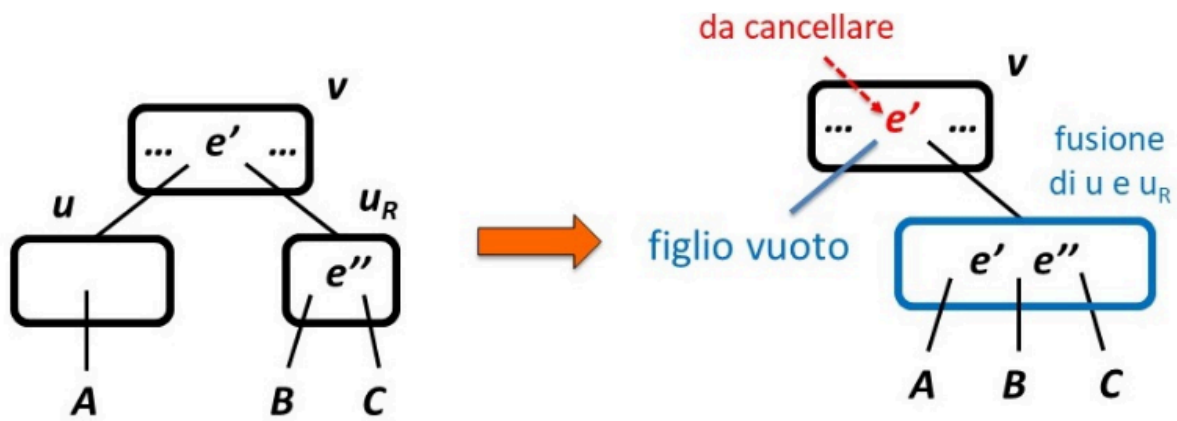
E' una situazione simmetrica alla precedente.

CASO 4: u ha un fratello u_L a sx che è un 2-node.



In questo caso Delete *propaga* l'underflow verso l'*alto* se la rimozione di e' genera underflow in v : in tal caso si evoca $Delete(e', v)$.

CASO 5: u ha un fratello u_R a dx che è un 2-node.



E' una situazione simmetrica alla precedente.

🔗 RAGIONAMENTO

Sicuramente u ha dei *fratelli*:

- Se il nodo è *radice*, le operazioni da svolgere sono *banali*.
- Se a destra o sinistra ha un fratello *3-node* o *4-node* la rimozione si conclude con una semplice rotazione.
- Se *tutti* i fratelli sono *2-node* (e ne ha sicuramente uno a dx o sx) allora è necessario (potenzialmente) *propagare* l'underflow fino a *ricondursi* ad un *caso più semplice*.

COMPLESSITA'

Per una *mappa* con n entry implementata tramite un *(2,4)-Tree* la complessità di remove è:

$$\Theta(\log n)$$

Infatti la complessità è *dominata* da:

- MWTreeSearch: $\Theta(\log n)$ operazioni.
- Eventuale *ricerca* dell'entry ad altezza 1 da sostituire al posto di e se questa è ad altezza > 1 : $\Theta(\log n)$ operazioni.
- Delete: algoritmo ricorsivo.

Per quanto riguarda Delete:

- $\leq h$ *invocazioni ricorsive* perchè parte da un nodo ad altezza 1 e viene invocato lungo il percorso da questo alla *radice* al più *una volta per livello*.
- Ogni invocazione ricorsiva richiede $\Theta(1)$ operazioni.

Quindi la complessità di Delete è proporzionale all'altezza ($\Theta(\log n)$), e così anche quella di remove.

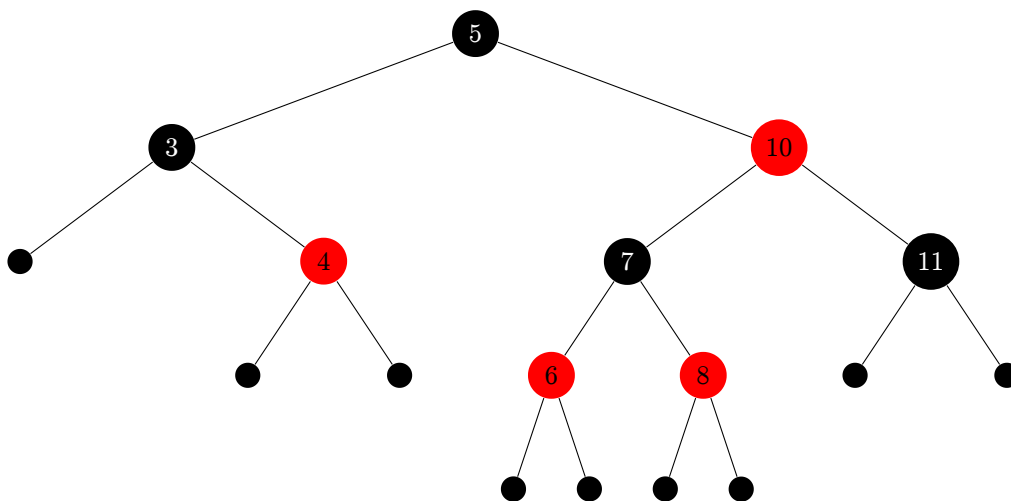
RED-BLACK TREE

I **Red-Black Tree** sono alberi binari di ricerca che, a *differenza* del caso generale, hanno *altezza* sempre *logaritmica* nel numero di nodi.

RED-BLACK TREE

Un red-black tree T è un ABR i cui nodi hanno *colore rosso* o *nero* e in cui valgono le seguenti proprietà:

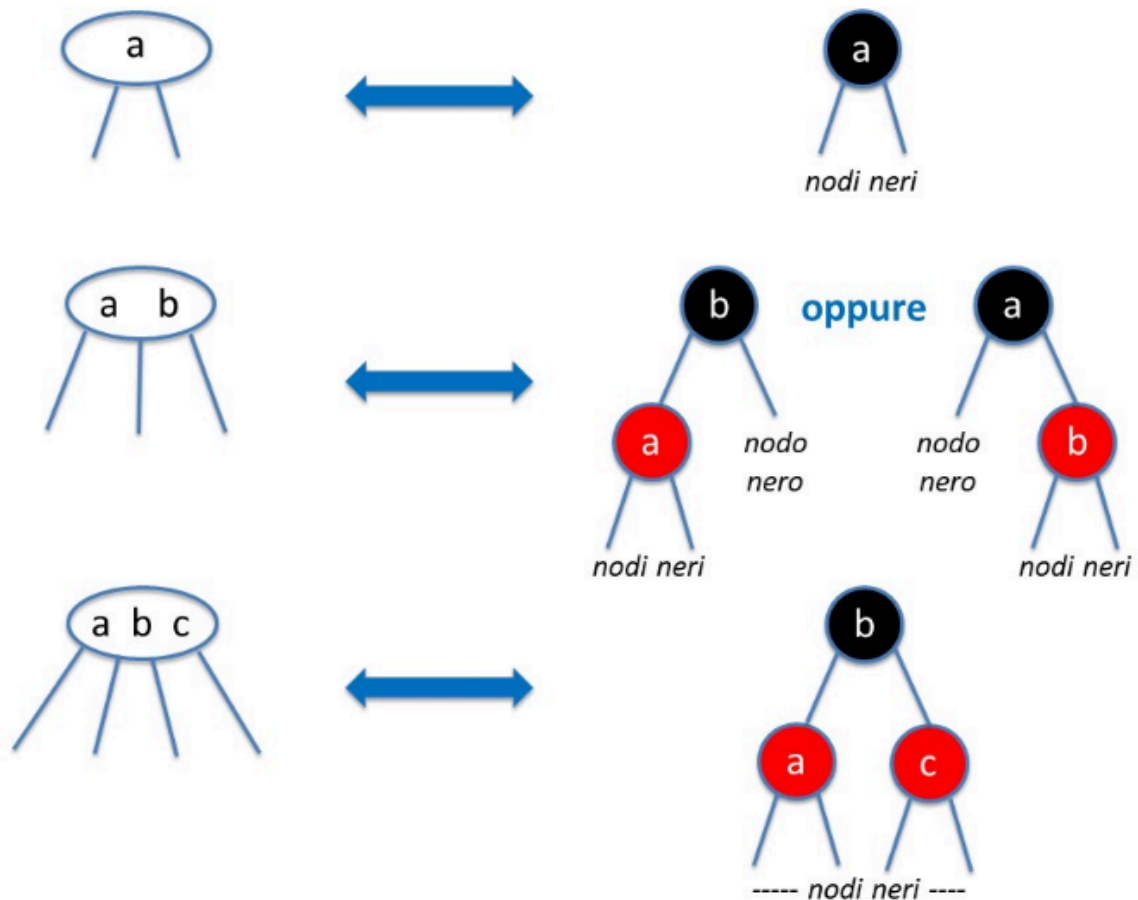
1. La *radice* è *nera* (Root property).
2. Le *foglie* sono *neri* (External property).
3. I *figli* di un *nodo rosso* sono *neri* (Red property).
4. Tutte le *foglie* hanno la stessa *black depth*, ovvero lo stesso numero di antenati propri neri (Depth property).



LEGAME CON I (2,4)-TREE

E' possibile *trasformare* un (2,4)-Tree in *Red-Black Tree* e viceversa, applicando le seguenti trasformazioni dalla radice verso le foglie:

1. Ad un *1-node* corrisponde un *nodo nero*.
2. Ad un *2-node* con entry a, b corrisponde una delle seguenti possibilità:
 - *nodo nero* con entry b e figlio *rosso* con entry a .
 - *nodo nero* con entry a e figlio *rosso* con entry b .
3. Ad un *3-node* con entry a, b, c corrisponde un *nodo nero* con entry b e *figli rossi* con entry a e c .



MAPPE CON RED-BLACK TREE

Valgono le seguenti proprietà (le cui dimostrazioni, seppur semplici, non studiamo):

- Un *Red-Black Tree* contenente n entry ha *altezza* $\Theta(\log n)$.
E' una diretta conseguenza delle *trasformazioni*: passando da un (2,4)-Tree ad un Red-Black tree *l'altezza al più raddoppia*, nel caso opposto *al più dimezza*.
- I *metodi* get, put, remove della mappa possono essere implementati in *Red-Black Tree* con complessità $\Theta(\log n)$ al caso pessimo.

🔗 OSSERVAZIONE

L'implementazione dei metodi della mappa risulta *efficiente* in pratica: a parte TreeSearch, per tutti e tre i metodi le *altre operazioni* sono in *numero costante*.
In *Java* la classe `TreeMap<k,V>` implementa una mappa tramite Red-Black Tree.

MULTIMAPPE

MULTIMAPPA

Una **multimappa** è una **mappa** che **ammette** la presenza di > 1 entry con la **stessa chiave**.

Questa differenza richiede una modifica della specifica dei metodi caratterizzanti:

- $\text{get}(k)$: restituisce una **collezione** (eventualmente vuota) con **tutti** i valori associati alle entry con chiave k .
- $\text{put}(k,v)$: inserisce **sempre** una nuova entry (k,v) **senza intaccare** altre entry con chiave k già presenti. Non restituisce alcun output.
- $\text{remove}(k,v)$: rimuove **una** entry con chiave k e valore v , se tale entry **esiste**. Restituisce un **boolean**: true se si è rimossa la entry, false altrimenti.

IMPLEMENTAZIONE

Una **multimappa** può essere implementata tramite una **mappa** in cui le **entry** sono costituite da coppie (k, L_k) dove k è una **chiave** ed L_k una **lista**, non vuota, dei valori **associati alla chiave**.

Se $L_k = \{v_1, v_2, \dots, v_l\}$ allora (k, L_k) rappresenta in modo compatto le l entry $(k, v_1), \dots, (k, v_l)$.

Per cui:

- $\text{get}(k)$: se **esiste** una entry k, L_k **restituisce** L_k , altrimenti restituisce una **collezione vuota**.
- $\text{put}(k,v)$: se **esiste** una entry (k, L_k) si **aggiunge** semplicemente il valore v a L_k , altrimenti si aggiunge la entry $(k, L_k = \{v\})$ alla struttura.
- $\text{remove}(k,v)$:
 1. Se esiste una entry (k, L_k) ed $L_k = \{v\}$ si **rimuove** la entry (k, L_k) e si restituisce true.
 2. Se esiste una entry (k, L_k) ed L_k contiene v ed **altri valori**, si **rimuove** v da L_k e si restituisce true.
 3. In tutti gli **altri casi** si restituisce false, **senza modificare** la multimappa.

ATTENZIONE

Nel caso esistano **più copie** della entry (k,v) da rimuovere, si può decidere di rimuoverne **una sola** o **tutte**.

INDICI PRIMARI E SECONDARI

Nelle *basi di dati* l'accesso ai dati è reso *efficiente* dall'utilizzo di *indici primari* e *secondari*.

- **Indice primario**: struttura di accesso ad una collezione di entry basata su una *chiave* che *non ammette duplicati* (es. numero di matricola di uno studente), realizzato tramite una *mappa*.
- **Indice secondario**: struttura di accesso ad una collezione di entry basata su una *chiave* che *ammette duplicati* (es. cognome studente), realizzato tramite una *multimappa*.

RIEPILOGO

COMPLESSITA' MAPPA

Si consideri una *mappa* con n entry.

METODO	Tab. Hash	ABR	(2,4)-Tree/RB-Tree
get(k)	$\Theta(1 + \lambda)$	$\Theta(h)$	$\Theta(\log n)$
put(k,v)	$\Theta(1 + \lambda)$	$\Theta(h)$	$\Theta(\log n)$
remove(k)	$\Theta(1 + \lambda)$	$\Theta(h)$	$\Theta(\log n)$

NOTA BENE:

- Per l'ABR, h può assumere valori *compresi fra* $\Theta(\log n)$ e $\Theta(n)$.
- La complessità per la tabella hash è al *caso medio*.

COMPLESSITA' MULTIMAPPA

Si consideri una *multimappa* con n chiavi distinte.

METODO	Tab. Hash	ABR	(2,4)-Tree/RB-Tree
get(k)	$\Theta(1 + \lambda)$	$\Theta(h)$	$\Theta(\log n)$
put(k,v)	$\Theta(1 + \lambda)$	$\Theta(h)$	$\Theta(\log n)$
remove(k)	$\Theta(s + 1 + \lambda)$	$\Theta(s + h)$	$\Theta(s + \log n)$

NOTA BENE:

- Il termine s nella complessità di *remove* indica il massimo numero di *entry con la stessa chiave*.
- Nelle complessità per la tabella hash, il termine $1 + \lambda$ si riferisce al *tempo medio* di *ricerca della chiave*, mentre nel caso di *remove*, il termine s per la ricerca della entry

è al *caso pessimo*.